
PasswordStore Audit Report

Prepared by: Riel

Date: June 2026

Protocol Summary

PasswordStore is a smart contract designed to allow a single owner to store and retrieve a private password. Only the owner should be able to set and access the password.

Disclaimer

The Riel team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

	High	Medium	Low
High	H	H/M	M
Medium	H/M	M	M/L
Low	M	M/L	L

Audit Details

- **Commit Hash:** [INSERT COMMIT HASH]
- **Scope:** `./src/PasswordStore.sol`
- **Solc Version:** 0.8.18
- **Chain:** Ethereum

Roles

- **Owner:** The user who can set the password and read the password.
- **Outsiders:** No one else should be able to set or read the password.

Executive Summary

Severity	Count
High	2
Medium	0
Low	0
Informational	1
Gas	0

Findings

High

[H-1] PasswordStore::setPassword lacks access control, allowing any user to overwrite the owner's password **Description:** The `PasswordStore::setPassword` function is intended to allow only the contract owner to set a new password. However, the function has no access control modifier (such as `onlyOwner`) or manual `msg.sender` check. As a result, any external caller can invoke `setPassword` and overwrite the owner's password.

Impact: Any malicious user can overwrite the owner's stored password, completely breaking the protocol's core guarantee of owner-only password management. The contract becomes useless for its intended purpose of private password storage, and the owner loses control over their own stored data.

Proof of Concept:

```
1 function test_anyoneCanSetPassword() public {
2     vm.prank(address(1)); // non-owner caller
3     passwordStore.setPassword("hacked");
4 }
```

```
5     vm.prank(owner);
6     string memory actual = passwordStore.getPassword();
7     assertEq(actual, "hacked");
8 }
```

Recommended Mitigation: Add an access control check to restrict `setPassword` to the contract owner only:

```
1 function setPassword(string memory newPassword) external {
2     if (msg.sender != s_owner) {
3         revert PasswordStore__NotOwner();
4     }
5     s_password = newPassword;
6     emit SetNetPassword();
7 }
```

[H-2] Storing the password on-chain makes it visible to anyone, rendering the protocol’s privacy guarantee broken **Description:** The `PasswordStore : s_password` variable is declared as `private`, but this only restricts access from other smart contracts at the Solidity level. All on-chain storage is publicly readable by anyone who queries the blockchain directly. This means the “private” password can be retrieved by reading the contract’s storage slot.

Impact: There is no way to store truly private data on-chain. The core protocol promise — that the password is only visible to the owner — is fundamentally broken regardless of the access control fix in H-1. Even if `setPassword` is properly restricted, anyone can still read the password.

Proof of Concept:

1. Start a local Anvil chain and deploy `PasswordStore`
2. Read storage slot 1 directly:

```
1 cast storage <CONTRACT_ADDRESS> 1 --rpc-url http://127.0.0.1:8545
```

3. Convert the hex output to string:

```
1 cast parse-bytes32-string <HEX_OUTPUT>
```

This returns the stored password in plaintext.

Recommended Mitigation: Encrypt the password off-chain before storing it on-chain. However, this introduces key management complexity. Fundamentally, a blockchain is not a suitable medium for storing data that must remain confidential.

Informational

[I-1] The PasswordStore::getPassword natspec indicates a parameter that doesn't exist

Description: The natspec for `getPassword` documents a `@param newPassword`, but the function signature takes no parameters. This is a copy-paste error from `setPassword`.

Impact: Natspec is incorrect, causing confusion for developers or auditors reading the codebase.

Recommended Mitigation: Remove the `@param newPassword` line from `getPassword` natspec.